מטלה 3 — אתר בהשראת FocusFlow

מטרת המטלה

במטלה הזו תבנו אתר Full-Stack שמודד, דרך מצלמת הרשת, עד כמה המשתמש מרוכז, ומגיב לכך. הרעיון לקוח מפרויקט בשם **FocusFlow**: אתר לימוד שבו כשהקשב יורד, האתר שואל שאלה, מציע הפסקה, או מחזיר את הצופה לקטע שהחמיץ.

אתם לא משחזרים אותו אלא בונים אתר משלכם ברוח דומה, שבהמשך יורחב לפרויקט האישי הסופי שלכם, גדול יותר בהיקפו. המטלה נעשית **בקבוצות של 2 עד 3 מגישים** (לא לבד). את החלקים ולוח הזמנים נסביר בכיתה.

בקצרה: מה חובה ומה רשות

סוג	מה זה כולל
חובה	מערכת משתמשים (הרשמה, התחברות, התנתקות). ספריית תוכן אישית עם סוג תוכן אחד לפחות. סשן עם מצלמה, מודל שמודד משהו על המשתמש (מומלץ מאוד: מודל הקשב שקיבלתם) ומדד שמוצג ונשמר. הודעה כשאין פנים בתמונה. כל משתמש רואה רק את התוכן שלו. כל המערכת ב-Docker (שלושה קונטיינרים). קובץ `PROJECT_INFO.docx` מלא. כל זה שווה **100 נקודות** (ראו "ניקוד")
קבוע	השרת נכתב ב-Python, הלקוח על פורט 3000, השרת על פורט 8000 עם `GET /health`, והגשה כקובץ `assignment3.zip`
בחירה חופשית	כלים (ההמלצה: FastAPI לשרת, MongoDB למסד הנתונים), הלקוח בכל טכנולוגיית ווב (React, JavaScript רגיל, TypeScript ועוד), עיצוב ושפת הממשק, סוגי התוכן, איפה רץ המודל (ההמלצה: בדפדפן), מה קורה כשהקשב יורד, ושמות של נתיבים ושדות. גם המודל והמדד: אפשר להשתמש במודל הקשב שקיבלתם (מומלץ מאוד), לבנות מודל משלכם, או למדוד משהו אחר מקשב, כל עוד מסבירים זאת ב-`PROJECT_INFO.docx`
רשות (בונוס)	כל השאר: פרופיל, מחיקת חשבון, אישור באימייל, אבטחה מוסברת, כמה סוגי תוכן, תמיכה בטלפון, אנליטיקה, עבודה עם המודל, Gemini, וכן תוסף לדפדפן (Plugin) שמזכה ב-30 נקודות

כל פרק בהמשך מפרט אחת מהשורות האלה. מה שלא מופיע בשורת החובה אינו חובה.

⚠ כלל הברזל: האתר צריך להסביר את עצמו

הבדיקה נעשית על ידי סוכן AI שנכנס לאתר בדפדפן ומשתמש בו כמו משתמש חדש: קורא את מה שמוצג, מחפש איך להירשם, להוסיף תוכן ולהתחיל סשן, לוחץ ומקליד. הוא לא קורא את הקוד ולא מקבל הוראות על הממשק, ולכן הממשק צריך להיות ברור למי שמגיע אליו בפעם הראשונה (ראו "קריאות ושימושיות" בפרק הלקוח).

- כשהסוכן לא מצליח להשלים שלב, הוא מסמן אותו ואנחנו בודקים ידנית. לא מפסידים נקודות רק כי הוא התקשה, אבל ממשק לא ברור עלול להוריד נקודות בסעיף הבהירות.
- הסוכן משתמש במצלמה וירטואלית (OBS) שמנגנת סרטונים מוקלטים של אדם ממוקד ושל אדם לא ממוקד. האתר צריך לעבוד איתה כמו עם מצלמה אמיתית.

קבצים מצורפים

- **models** — תשעה קבצי ONNX של המודלים (v1 עד v4_66). הסבר מלא בפרק על המודל.
- **diagrams** — דיאגרמות UML של הפרויקט המקורי, כדוגמה לדיאגרמות שתכינו בעצמכם.
- **docker-compose** לדוגמה — מהפרויקט המקורי, להשראה בלבד.
- **env.example.** — תבנית לקובץ `.env` שמגישים.
- **PROJECT_INFO.docx** — תבנית לקובץ המידע על הפרויקט, שממלאים ומגישים.

טכנולוגיות

- **חובה:** השרת נכתב ב-Python (בכל ספרייה), וכל המערכת רצה ב-Docker בשלושה קונטיינרים (לקוח, שרת ומסד נתונים) שמחוברים בקובץ docker-compose.
- **מומלץ מאוד:** FastAPI לשרת (תיעוד אינטראקטיבי ב-`/docs` בחינם) ו-MongoDB למסד הנתונים (התמונה הרשמית). לכלים האלה יש הרבה תיעוד ודוגמאות, וכלי AI מכירים אותם היטב.
- **חופשי:** הלקוח יכול להיות כתוב בכל טכנולוגיית ווב: React (למשל עם Vite), JavaScript רגיל, TypeScript, Vue, Svelte ועוד. גם בשרת ובמסד הנתונים מותר לבחור אחרת (למשל Flask, Django או PostgreSQL), כל עוד השרת ב-Python והדברים הקבועים נשמרים.

איך המערכת בנויה

רכיב	תפקיד	איפה רץ
לקוח (Frontend)	ממשק המשתמש, הצגת התוכן, הפעלת המצלמה, חילוץ נקודות פנים בדפדפן באמצעות MediaPipe, והרצת מודל הקשב (מומלץ) באמצעות onnxruntime-web	קונטיינר Docker, פורט 3000
שרת (Backend)	מערכת משתמשים, שמירת תוכן, סשנים וציוני קשב (ובתוספת בונוס: הרצת המודל בשרת)	קונטיינר Docker, פורט 8000
מסד נתונים	משתמשים, תוכן, סשנים וציוני קשב	קונטיינר Docker (המלצה: MongoDB), עם volume לשמירת הנתונים

זרימה של סשן צפייה:

1. המשתמש נכנס לאתר, מתחבר, ובוחר פריט תוכן מהספרייה שלו (למשל סרטון, טקסט או דף אינטרנט).
2. הלקוח פותח סשן חדש בשרת, ומבקש מהמשתמש הסכמה להפעלת המצלמה.
3. הדפדפן מחלץ מהמצלמה נקודות פנים של MediaPipe. הווידאו עצמו לא עוזב את הדפדפן.
4. הלקוח אוסף חלון של 100 פריימים (כעשר שניות) ומריץ עליו את המודל בדפדפן.
5. הלקוח שולח את הציון לשרת, והשרת שומר אותו במסד הנתונים.
6. הלקוח מציג את הציון מיד, ואם הוא נמוך מופיעה התראה (או שאלה, בהמשך הפרויקט).
7. בסיום המשתמש סוגר את הסשן, ויכול לראות אותו בהיסטוריה.

השרת (Backend): מה הוא חייב לספק

השרת חושף API שמחזיר ומקבל JSON ומשרת את הלקוח שלכם. מומלץ שכל הנתיבים יתחילו ב-`/api`. נתיב אחד חייב להיות בדיוק כפי שהוא: `GET /health`, שמחזיר 200 כשהשרת מוכן. זמנים מוחזרים בפורמט ISO 8601 (UTC), ומזהים (`id`) הם מחרוזות.

1. מערכת משתמשים

מערכת משתמשים מלאה, כמו באתר אמיתי: הרשמה, התחברות והתנתקות. פרופיל, החלפת סיסמה ומחיקת חשבון הם בונוס.

- **הרשמה:** אימייל, סיסמה ושם תצוגה. האימייל ייחודי (בלי הבדל בין אותיות גדולות לקטנות), והסיסמה באורך 8 תווים לפחות.
- **התחברות:** בהתחברות מוצלחת השרת מחזיר טוקן גישה (למשל JWT) עם תוקף מוגבל, והלקוח שולח אותו בכל בקשה בכותרת `Authorization: Bearer <token>`.
- **הרשאות:** נתיב שמסומן "מחובר" דורש טוקן תקין, אחרת הוא מחזיר 401. כל משתמש רואה ומשנה רק את הנתונים שלו: בקשה למשאב של משתמש אחר מחזירה 404.
- **אבטחה בסיסית:** הסיסמה נשמרת רק כ-Hash באמצעות אלגוריתם ייעודי (bcrypt או argon2), ולא מוחזרת בשום תגובה. כתובת מסד הנתונים וכל סוד אחר מגיעים ממשתני סביבה, לא מהקוד. איך עשיתם את זה מסבירים ב-`PROJECT_INFO.docx`, והסבר טוב מזכה בבונוס אבטחה.

2. תוכן, סשנים ומודל

- **תוכן:** כל משתמש מנהל ספרייה אישית. פריט תוכן הוא כל דבר שאפשר לצפות בו או לקרוא תוך כדי מדידת קשב, ואתם בוחרים אילו סוגים האתר תומך בהם. חובה לתמוך לפחות בסוג אחד, ולציין אותו ב-`PROJECT_INFO.docx`. דוגמאות: סרטון YouTube (קישור), טקסט ארוך שמודבק, דף אינטרנט שהמשתמש מדביק את הקישור שלו, קובץ PDF, קובץ וידאו שמועלה, או כל סוג אחר שהמצאתם. שימו לב: אתרים רבים חוסמים הצגה בתוך אתר אחר (iframe), ולכן הצגת דף חיצוני בדרך כלל דורשת לשלוף ולהציג את הטקסט שלו בעצמכם, או לפתוח אותו בחלון נפרד.
- **סשן:** פעם אחת של צפייה או קריאה בפריט תוכן. בתוך הסשן נשמרות מדידות (`readings`): מיקום בתוכן וציון. עם מודל הקשב הציון הוא בין 0 ל-1. אם הסשן אצלכם שונה, למשל מתחיל אוטומטית עם פתיחת פריט או לא קשור לפריט תוכן, תארו אותו ב-`PROJECT_INFO.docx`.
- **מדידה ומודל:** מומלץ מאוד להריץ את אחד ממודלי הקשב שקיבלתם (ראו הפרק על המודל). אפשר גם לבנות מודל משלכם או לבחור מודל אחר ולמדוד משהו אחר מקשב, למשל עייפות או מצב רוח. זו בחירה חופשית, וחשוב לתאר ב-`PROJECT_INFO.docx` מה נמדד, באיזה טווח ואיך המודל נבנה. הדרך המומלצת להריץ מודל היא להריץ אותו בדפדפן: הלקוח מחשב את הציון ושולח אותו לשרת לשמירה, וכך נקודות הפנים לא עוזבות את המכשיר. הרצה בשרת אפשרית, אבל מסובכת יותר ואיננו ממליצים עליה. בשני המקרים השרת שומר את הציון.

ה-API המומלץ

הסוכן לא פונה ל-API ישירות, אלא רק דרך הלקוח שבניתם. לכן הטבלה היא המלצה ותיאור של ההתנהגות הנדרשת, ולא חוזה: אפשר לשנות נתיבים ושמות שדות, כל עוד הלקוח והשרת עובדים יחד וההתנהגות נשמרת (הרשאות, שגיאות ושמירת נתונים). בעמודת ההרשאה, "פתוח" אומר שאין צורך בטוקן, ו"מחובר" אומר שהטוקן נדרש.

נתיב	הרשאה	גוף הבקשה	סטטוס	תגובה
`GET /health`	פתוח	—	200	`{"status":"ok"}`
`POST /api/auth/register`	פתוח	`email`, `password`, `display_name`	201	`id`, `email`, `display_name` 409 אם האימייל קיים. 422 אם הקלט לא תקין
`POST /api/auth/login`	פתוח	`email`, `password`	200	`access_token`, `token_type` (הערך "bearer"), `expires_in` (בשניות) 401 אם הפרטים שגויים
`POST /api/auth/logout`	מחובר	—	204	—
`GET /api/users/me`	מחובר	—	200	`id`, `email`, `display_name`, `created_at`
`PATCH /api/users/me`	מחובר	`display_name`	200	פרטי המשתמש המעודכנים
`POST /api/users/me/password`	מחובר	`current_password`, `new_password`	204	400 אם הסיסמה הנוכחית שגויה
`DELETE /api/users/me`	מחובר	—	204	מוחק גם את כל הנתונים של המשתמש
`POST /api/content`	מחובר	`title`, `type` (סוג התוכן לפי בחירתכם, למשל text, youtube או web), `source` (קישור, או גוף הטקסט)	201	`id`, `title`, `type`, `source`, `created_at` 422 אם הסוג לא נתמך או שהמקור לא תקין
`GET /api/content`	מחובר	—	200	רשימת פריטי התוכן של המשתמש

נתיב	הרשאה	גוף הבקשה	סטטוס	תגובה
`GET /api/content/{id}`	מחובר	—	200	הפריט. 404 אם לא קיים או שאינו של המשתמש
`DELETE /api/content/{id}`	מחובר	—	204	—
`POST /api/sessions`	מחובר	`content_id`	201	`id`, `content_id`, `started_at`
`POST /api/sessions/{id}/readings`	מחובר	`position` (שניות בסרטון, או מיקום בטקסט), ו-`score` אם המודל רץ בלקוח, או `landmarks` (מערך 100×478×3) אם הוא רץ בשרת	201	`position`, `score`, `created_at`. 422 אם הקלט לא תקין. 409 אם הסשן נסגר
`POST /api/sessions/{id}/end`	מחובר	—	200	`id`, `started_at`, `ended_at`, `reading_count`, `average_score`
`GET /api/sessions`	מחובר	—	200	רשימת סשנים: סיכום לכל אחד, כולל `content_id` ו-`average_score`
`GET /api/sessions/{id}`	מחובר	—	200	סיכום הסשן, ובנוסף `readings`: רשימה של `position`, `score`, `created_at`

שגיאות: כל שגיאה מחזירה JSON בצורה `{"detail": "message"}` עם קוד הסטטוס המתאים: 400 בקשה שגויה, 401 לא מחובר, 404 לא נמצא, 409 התנגשות, 422 קלט לא תקין.

מה נשמר במסד הנתונים

את הסכמה אתם מתכננים, אבל חייבים להיות בה לפחות שלושה סוגי מידע:

- **משתמשים:** אימייל (ייחודי), Hash של הסיסמה, שם תצוגה, תאריך יצירה.
- **תוכן:** הבעלים, כותרת, סוג, מקור (קישור או טקסט), תאריך יצירה.
- **סשנים ומדידות:** המשתמש, פריט התוכן, זמן התחלה וסיום, ורשימת המדידות (מיקום, ציון, זמן).

הלקוח (Frontend): מה הוא חייב לספק

הלקוח הוא אתר שרץ בדפדפן על פורט 3000. אין צורך בסימונים מיוחדים בקוד: הסוכן מנווט לפי מה שמוצג על המסך, כמו משתמש רגיל. לכן הטבלה מתארת מה המשתמש צריך להיות מסוגל לעשות בכל מסך, ולא איך לקרוא לו.

מסך	מה המשתמש עושה בו	מה חייב להיות מוצג
הרשמה	פותח חשבון חדש: אימייל, סיסמה ושם תצוגה	שדות עם תוויות ברורות, כפתור הרשמה וקישור למי שכבר רשום. הודעה מובנת אם האימייל קיים או שהסיסמה קצרה מדי. אחרי הצלחה עוברים להתחברות
התחברות	נכנס עם אימייל וסיסמה	שדות ברורים, כפתור כניסה וקישור להרשמה. הודעה ברורה כשהפרטים שגויים. אחרי הצלחה עוברים לספרייה
ספרייה	רואה את התוכן שלו, מוסיף פריט (כותרת, סוג, וקישור, טקסט או קובץ, לפי הסוגים שתומכים בהם), פותח פריט ומוחק פריט	רשימה עם כותרות הפריטים, הודעה כשהספרייה ריקה, טופס הוספה ברור, ואישור לפני מחיקה
צפייה או קריאה	צופה בתוכן או קורא אותו (סרטון, טקסט, דף ועוד), מפעיל מצלמה, ומסיים סשן	כפתור לפתיחת סשן, כפתור להפעלת המצלמה עם הסבר על מה נאסף, ציון הקשב הנוכחי שמוצג בבירור, הודעה כשלא זוהו פנים, התראה כשהקשב יורד, וכפתור לסיום הסשן
היסטוריה	רואה סשנים קודמים ופותח אחד מהם	רשימה עם תאריך, שם התוכן וציון ממוצע. בלחיצה על סשן מוצגות המדידות שלו

מסך	מה המשתמש עושה בו	מה חייב להיות מוצג
פרופיל (בונוס)	מעדכן שם תצוגה, מחליף סיסמה ומוחק חשבון	הפרטים הנוכחיים, טפסים ברורים, ואישור מפורש לפני מחיקת החשבון

התנהגות הלקוח

- **ניווט והרשאות:** בכל מסך של משתמש מחובר יש קישורים לספרייה ולהיסטוריה, שם התצוגה וכפתור התנתקות. משתמש שלא מחובר מועבר להתחברות, וכך גם משתמש שהשרת החזיר לו 401. הטוקן נשמר כך שרענון הדף לא מתנתק.
- **פריט שלא קיים או לא שלי:** הלקוח מציג הודעה ברורה "לא נמצא", בלי מידע על הפריט.
- **הצגת תוכן:** כל סוג תוכן מוצג בתצוגה שמתאימה לו, ליד אזור המצלמה: למשל סרטון YouTube בנגן מוטמע (YouTube IFrame), טקסט בתצוגת קריאה, ודף אינטרנט בתצוגת קריאה שהאתר שלכם בנה.
- **הסכמה לפני מצלמה:** המצלמה נדלקת רק אחרי שהמשתמש לחץ על כפתור ייעודי ואישר. אם הוא מסרב, אפשר להמשיך לצפות בלי מדידה, והאתר מסביר מה קרה.
- **מדידה:** MediaPipe רץ בדפדפן, ומחלץ נקודות פנים בקצב 10 פריימים לשנייה. כל כמה שניות מריצים את המודל על חלון נע של 100 פריימים, והציון נשלח לשרת לשמירה. הציון הראשון מופיע אחרי כעשר שניות, והסוכן ימתין לו. אם לא זוהו פנים, מציגים "לא זוהו פנים" ולא מריצים את המודל. המצלמה יכולה להיות כל מצלמה שהדפדפן מזהה, כולל וירטואלית.
- **הצגת הציון:** הציון האחרון מוצג בצורה בולטת, עם תווית שמסבירה מה הוא. כשהוא מצביע על בעיה (עם מודל הקשב, למשל מתחת ל-0.4) מופיעה התראה (בהמשך הפרויקט אפשר להחליף אותה בשאלה או בהתערבות אחרת).
- **כתובת השרת:** הלקוח פונה לשרת דרך כתובת שניתנת להגדרה (משתנה סביבה או קובץ הגדרות), לא כתובת קבועה בקוד. זה יחסוך בעיות כשתעברו לענן.

קריאות ושימושיות

אתר טוב הוא כזה שמשתמש חדש מבין בלי הסבר:

- כפתורים וקישורים עם טקסט ברור, לא רק אייקון.
- כל פעולה מגיבה: רואים שהיא בתהליך, שהצליחה או שנכשלה, והודעות שגיאה מופיעות ליד הבעיה בשפה מובנת.
- טקסט קריא עם ניגודיות טובה, מסך לא עמוס, ואם הממשק בעברית, הוא מוצג מימין לשמאל.
- משתמש חדש מצליח להירשם, להוסיף תוכן, להתחיל סשן ולראות היסטוריה בלי הוראות. עבודה טובה על מסך מחשב מספיקה, ותמיכה בטלפון היא בונוס.

Docker ו-docker-compose

- קובץ `docker-compose.yml` אחד מפעיל את כל המערכת בפקודה `docker compose up`, בלי שלבים ידניים. הלקוח זמין ב-`http://localhost:3000`, והשרת ב-`http://localhost:8000` (בדיקת תקינות: `/health`).
- מסד הנתונים הוא הקונטיינר השלישי (למשל התמונה הרשמית `mongo`), עם volume כדי שהנתונים יישמרו בין הרצות. השרת מתחבר אליו בשם השירות ב-Compose (למשל `mongodb://mongo:27017`) ולא ב-`localhost`, כי בתוך קונטיינר `localhost` הוא הקונטיינר עצמו.
- ה-Compose מושך תמונות (`image:`) ציבוריות, למשל מ-Docker Hub או מ-GHCR, ולכן אנחנו לא צריכים את הקוד כדי להריץ. התמונות חייבות להיות בנויות ל-`linux/amd64`: מחשבי Apple Silicon בונים כברירת מחדל ל-arm64, ולכן בנו עם `docker buildx build --platform linux/amd64`. קבצי המודל נמצאים בתוך התמונה ולא יורדים בזמן ההרצה.

קובץ .env

מגישים גם קובץ `.env` עם הערכים המקומיים של המערכת (כתובת המסד, ומפתח Gemini ריק). תבנית מצורפת: `.env.example`.

- שירות חיצוני שדורש מפתח יכול להיות רק Gemini, ורק אם בחרתם בו. את המפתח משאירים **ריק** בהגשה, והאתר חייב לעבוד גם בלעדיו (מה שלא זמין נכשל בצורה מנומסת). אותו כלל חל על הגדרות של תוספות רשות, כמו שירות דואר לאישור הרשמה.
- אין בקובץ סודות אמיתיים, ולכן אפשר להגיש אותו כמו שהוא. אל תעלו ל-GitHub סוד אמיתי, ואם זה קרה, החליפו אותו מיד. מסד נתונים בענן (למשל MongoDB Atlas) ייכנס רק בהמשך.

קובץ המידע על הפרויקט (PROJECT_INFO.docx)

מסמך Word קצר בעברית, עם טבלה לכל נושא: בכל שורה שם שדה והסבר, ואתם כותבים את התשובה בעמודה האחרונה. הסוכן קורא אותו לפני הבדיקה, וכך הוא מכיר את האתר שלכם: סוג התוכן העיקרי, תוכן לדוגמה לבדיקה, שפת הממשק ואיך מגיעים לסשן.

הנושאים: פרטים כלליים, תוכן, טכנולוגיה (כולל איזה מודלים רצים אצלכם ואיפה, ואפשר להתפאר במה שמיוחד), שימוש באתר, הרצה, **אבטחה והגנה על נתונים**, מצב הפרויקט והפרויקט שלכם. את הדברים שאי אפשר לראות מהדפדפן, כמו אחסון סיסמאות וטיפול בטוקן, מסבירים בסעיף האבטחה, והסבר קונקרטי שם מזכה בבונוס אבטחה (ראו "ניקוד").

- מלאו כל שדה. מה שלא רלוונטי, כתבו `none`, ומה שעובד חלקית כתבו במפורש: זה מקל על הבדיקה.
- אל תכתבו בקובץ סיסמאות, מפתחות או כתובות התחברות. אל תשנו שמות של שדות ואל תמחקו שורות.
- אם הקובץ סותר את מה שמוצג באתר, מה שמוצג באתר קובע.

מה מגישים

קובץ zip אחד בשם `assignment3.zip`, עם שלושת הקבצים ברמה העליונה (לא בתוך תיקייה):

```
assignment3.zip
├── docker-compose.yml
├── .env
└── PROJECT_INFO.docx
```

- אין צורך בקוד המקור: הבדיקה מריצה את האתר דרך הדפדפן בלבד. הקפידו על השמות בדיוק כפי שהם כאן, כולל אותיות קטנות.
- מי שבנה תוסף לדפדפן (בונוס) מוסיף תיקייה בשם `extension` עם קבצי התוסף.

ניקוד הבסיס המשותף (100 נקודות)

הסוכן קורא קודם את `PROJECT_INFO.docx` ואז בודק את הסעיפים הבאים דרך הדפדפן בלבד, ובמקום שבו משהו אצלכם שונה (למשל הסשן) הוא פועל לפי מה שתיארתם שם. עברו עליהם בעצמכם לפני ההגשה, כאילו אתם משתמשים חדשים.

#	נושא	נקודות	מה בודקים
1	הפעלה	10	`docker compose up` מעלה את המערכת, `/health` מחזיר 200, והאתר נטען ב-`localhost:3000`
2	הרשמה	10	משתמש חדש נרשם בהצלחה. אימייל שכבר קיים וסיסמה קצרה מדי נדחים עם הודעה מובנת
3	התחברות	10	סיסמה שגויה מציגה הודעה ברורה, סיסמה נכונה מכניסה לאתר, ושם התצוגה מוצג

#	נושא	נקודות	מה בודקים
4	תוכן	14	הוספת פריט מהסוג העיקרי שציינתם ב-`PROJECT_INFO.docx`, עם תוכן הדוגמה שסיפקתם. הוא מופיע בספרייה ונפתח. סוגים נוספים שציינתם ינוסו גם הם
5	סשן ומדידה	26	פתיחת סשן, הפעלת המצלמה אחרי הסכמה וצפייה של כחצי דקה בסרטון מוקלט של אדם ממוקד. מופיע ציון שמתעדכן ומתנהג כפי שתיארתם ב-`PROJECT_INFO.docx` (עם מודל הקשב: בין 0 ל-1, וגבוה אצל אדם ממוקד). אחרי סיום הסשן הוא מופיע בהיסטוריה עם ציון ממוצע
6	חוסר פנים	8	כשהמצלמה מנגנת סרטון בלי פנים, מוצגת הודעה מתאימה והאתר ממשיך לעבוד
7	הפרדה בין משתמשים	8	משתמש שני נרשם ורואה ספרייה ריקה, ופתיחת כתובת של פריט של המשתמש הראשון מציגה "לא נמצא"
8	התנתקות	4	התנתקות מחזירה למסך ההתחברות
9	בהירות ושימושיות	5	כפתורים וטקסטים ברורים, הודעות שגיאה מובנות ומסלול ברור לכל פעולה
10	קבצי ההגשה	5	`assignment3.zip` במבנה הנכון, עם `docker-compose.yml`, `.env` ו-`PROJECT_INFO.docx` מלא
	סה"כ	**100**	

- בכל שלב מקבלים ניקוד מלא, חלקי או אפס, לפי כמה מהפעולות בו עובדות.
- שלב שהסוכן סימן כלא מצליח נבדק על ידינו ידנית, והניקוד נקבע לפי הבדיקה הידנית.

בונוס: תוספות רשות

תוספות רשות מזכות בנקודות רק אם הבסיס עובד. אפשר לצבור עד 20 נקודות מהטבלה, ועוד 30 מתוסף הדפדפן שבהמשך. **ציון מעל 100 מעוגל ל-100**, ולכן הבונוס משלים נקודות שהפסדתם ואינו מוסיף מעבר למקסימום. אין צורך להספיק הכול.

תוספת	נקודות	איפה מוסברת
פרופיל: עדכון שם תצוגה והחלפת סיסמה	עד 3	פרק הלקוח
מחיקת חשבון עם אישור, שמוחקת גם את כל הנתונים של המשתמש	עד 3	פרק השרת
אישור הרשמה באימייל (הגדרות הדואר נשארות ריקות בהגשה, והאתר עובד גם בלעדיהן)	עד 4	פרק ה-Docker
אבטחה מוסברת: סעיף האבטחה ב-`PROJECT_INFO.docx` מלא וקונקרטי, וההגנות שמתוארות בו נכונות	עד 5	PROJECT_INFO
הגנה נוספת על החשבון: כללי סיסמה חזקה, הגבלת ניסיונות התחברות, סוד חתימה לטוקנים (`JWT_SECRET`)	עד 4	PROJECT_INFO
תמיכה בכמה סוגי תוכן (למשל סרטון, טקסט ודף אינטרנט) עם תצוגה מתאימה לכל סוג	עד 3	פרק השרת
האתר נוח לשימוש גם במסך טלפון	עד 3	פרק הלקוח
דף אנליטיקה: קשב לאורך זמן ובין סשנים, או שימוש במשקלי Attention (v4_2 ומעלה)	עד 4	רעיונות לתוספות
עבודה מקורית עם המודל, בכל דרך: מודל שאימנתם בעצמכם, מדידה של משהו אחר מקשב, השוואת כמה מודלים, החלקת הציון, סף חכם להתערבות, שימוש בסיווג ל-5 רמות או במשקלי Attention	עד 4	פרק המודל
אתגר למתקדמים: הרצת המודל בשרת עם WebSocket שמזרים נקודות פנים וציונים בזמן אמת (מסובך, לא מומלץ למי שמתחיל)	עד 4	פרק המודל
תכונה שנעזרת ב-Gemini (שאלות, סיכום או משוב) שמתנהגת יפה גם בלי מפתח	עד 4	רעיונות לתוספות

ניקוד אבטחה: הסבר קונקרטי, עם שמות של אלגוריתמים, ספריות וערכים (למשל "bcrypt דרך ספרייה X" או "טוקן תקף 60 דקות"), מקבל ניקוד, והסבר כללי כמו "השתמשנו באבטחה" לא. אנחנו גם בודקים דרך הדפדפן מה שאפשר, כמו מה קורה אחרי התנתקות או בפתיחת פריט של משתמש אחר.

בונוס גדול: FocusFlow כ-Plugin (30 נקודות)

אתגר למי שרוצה לעבוד על משהו גדול: להביא את FocusFlow אל המקום שבו המשתמש כבר נמצא, בתור **תוסף אמיתי** (Plugin). שתי אפשרויות, ושתיהן מזכות באותם 30 נקודות:

- **תוסף לדפדפן** (Chrome, Manifest V3): מודד את הקשב של המשתמש בכל דף אינטרנט שהוא פותח, למשל דף YouTube או מאמר, ומציג ציון חי.
- **תוסף לאפליקציה אחרת**, למשל Zoom, Google Meet או Teams: מודד את הקשב של המשתמש בזמן שיחה או הרצאה, ומציג ציון חי או סיכום בסופה. כל פלטפורמה מגבילה אחרת את הגישה למצלמה ולממשק, ולכן כדאי לבדוק מראש מה אפשרי.

בשתי האפשרויות:

- התוסף מבקש הסכמה לפני שהוא מפעיל מצלמה, מודד **רק את המשתמש עצמו** (לא משתתפים אחרים), מחלץ נקודות פנים עם MediaPipe ומריץ את המודל אצל המשתמש (למשל עם `onnxruntime-web`). השרת והמסד רצים ב-Docker כמו בבסיס.
- **תחליף ללקוח או תוספת לו:** התוסף יכול להיות הלקוח כולו (הרשמה, התחברות, ספרייה, סשנים והיסטוריה בתוכו), או תוספת ללקוח רגיל על פורט 3000, שמתחברת לאותו שרת ולאותו חשבון, כך שסשנים שנמדדו בתוסף מופיעים גם באתר. אם הוא תחליף ללקוח, קונטיינר הלקוח על פורט 3000 יכול להציג דף שמסביר איך מתקינים אותו.
- **בדיקה:** התוסף נבדק על ידינו ידנית. אם הוא תחליף ללקוח, גם הבסיס נבדק דרכו, ולכן כל פעולות הבסיס צריכות לעבוד בו. אם הוא רץ באפליקציה שאין לנו גישה אליה, הסבירו איך בודקים אותו וצרפו סרטון הדגמה קצר.
- **הגשה:** תיקייה בשם `extension` בתוך ה-zip עם קוד התוסף. תוסף לדפדפן צריך להיות כזה שאפשר לטעון ב-Chrome ("Load unpacked"). בכל תוסף מסבירים ב-`PROJECT_INFO.docx` על איזו פלטפורמה הוא בנוי, איך מתקינים ואיך משתמשים בו.
- **שימו לב:** זה קשה ומגביל, והמצלמה וה-WebAssembly דורשים הגדרות ייחודיות בכל פלטפורמה. הבונוס ניתן רק אם הבסיס עובד ורק לתוסף שעובד בפועל. **חובה להתייעץ איתנו לפני שמתחילים**, כדי לוודא שהרעיון אפשרי ושנוכל לבדוק אותו.

המודל: מה הוא קורא ומה הוא מחזיר

הפרק מתאר את מודלי הקשב שמצורפים, ואנחנו ממליצים עליהם מאוד. מי שבוחר מודל אחר או מודל משלו יכול להשתמש בפרק כהשראה ולדלג על מה שלא רלוונטי לו.

המודל מקבל חלון של פריימים של נקודות פנים, ומחזיר ציון רציף של רמת הקשב (רגרסיה). הוא אומן על מאגרי הנתונים EngageNet ו-DAiSEE (סרטונים של אנשים צופים בתוכן ללמידה, עם תיוג של רמת קשב).

שדה	ערך
קלט	מערך בצורה `[batch, frames, 478, 3]`: חלון של פריימים, ובכל פריים 478 נקודות פנים של MediaPipe Face Mesh (כולל העיניים), עם שלוש קואורדינטות (x, y, z) לכל נקודה. מספר הנקודות חייב להיות בדיוק 478
אורך החלון	המודל אומן על 100 פריימים בקצב של 10 לשנייה, כלומר כעשר שניות. חלון באורך אחר עובד טכנית, אבל התוצאות לא מובטחות
נירמול	הנירמול של מרחק הפנים מהמצלמה כבר נמצא בתוך המודל. שולחים נקודות גולמיות של MediaPipe, בלי עיבוד נוסף
פלט ראשי	ציון בין 0 ל-1, כשגבוה יותר אומר קשב גבוה יותר
המרה לרמה	באימון הוגדרו הערכים: 0.05 הנבדק לא נמצא במצלמה, 0.30 לא קשוב, 0.50 קשב חלש, 0.70 קשוב, 0.95 קשוב מאוד

איך המודל אומן: MediaPipe

המודל אומן על נקודות שחולצו באמצעות MediaPipe Face Mesh, ולכן הוא מצפה בדיוק לנקודות כאלה:

- השתמשו ב-MediaPipe ולא בספרייה אחרת. הסדר והמשמעות של 478 הנקודות קבועים (למשל אינדקס 1 הוא קצה האף), והמודל מסתמך עליהם.
- בדפדפן אפשר להשתמש בחבילה `@mediapipe/tasks-vision` (FaceLandmarker), שמחזירה 478 נקודות לכל פריים. בפייתון, בחבילה `mediapipe`.
- באימון הפריימים נדגמו ב-10 לשנייה, ולפני החילוץ נחתך אזור הפנים (תמונה בגודל 256 פיקסלים עם מרווח של 20% סביב הפנים). כדאי לדגום באותו קצב, ולבדוק בניסוי אם חיתוך הפנים משפר את התוצאה אצלכם.

הרצת המודל בדפדפן (מומלץ)

בדפדפן מריצים את המודל עם `onnxruntime-web` (WebAssembly). בדקנו שכל תשעת המודלים רצים בו, וחלון אחד לוקח כ-50 אלפיות שנייה. נקודות הפנים, הציון והמודל נשארים כולם במכשיר של המשתמש. את קובץ ה-ONNX מגישים כקובץ סטטי של הלקוח, וטוענים אותו פעם אחת:

```
import * as ort from "onnxruntime-web";

const session = await ort.InferenceSession.create("/models/v4_2.onnx");
const inputName = session.inputNames[0];   // differs between versions

// frames: Float32Array of length 100 * 478 * 3 (x, y, z per landmark)
const input = new ort.Tensor("float32", frames, [1, 100, 478, 3]);
const outputs = await session.run({ [inputName]: input });
const score = outputs[session.outputNames[0]].data[0];   // the first output is the score
```

הרצת המודל בשרת (אפשרות למתקדמים, לא מומלצת)

בפייתון מריצים את המודל עם `onnxruntime` ו-`numpy`. טוענים אותו פעם אחת כשהשרת עולה, ולא בכל בקשה:

```
import numpy as np
import onnxruntime as ort

session = ort.InferenceSession("v4_2.onnx")
input_name = session.get_inputs()[0].name   # the input name differs between versions

window = np.zeros((1, 100, 478, 3), dtype=np.float32)
outputs = session.run(None, {input_name: window})
score = float(outputs[0][0][0])   # the first output is always the score
```

בגרסאות v1 עד v3 יש פלט אחד. ב-v4 ואילך יש גם סיווג לחמש רמות, ובגרסאות v4_2 ומעלה גם משקלי Attention.

גרסה	ארכיטקטורה	פלט
v1	GRU	ציון
v2	LSTM	ציון
v3	GRU עם Attention	ציון
v4	GRU עם Attention	ציון + סיווג לחמש רמות (לא נמצא במצלמה, לא קשוב, קשב חלש, קשוב, קשוב מאוד)
v4_2, v4_3, v4_4, v4_7, v4_66	אותה ארכיטקטורה של v4, עם משקלים שונים (תוצאות של ריצות אימון שונות)	ציון + סיווג + משקל Attention לכל פריים, שמראה אילו רגעים בחלון השפיעו הכי הרבה

- המודלים לא מושלמים. הם נבנו במהירות והדיוק שלהם מוגבל. הסבירו בהגנה על הפרויקט איזה מודל בחרתם, מה הוא מפספס, ואיך המערכת שלכם מתמודדת עם זה.
- הציון הוא הערכה סטטיסטית על חלון של כעשר שניות, לא קריאת מחשבות. אל תבנו החלטות חמורות על ציון בודד.
- נקודות הפנים הן מידע ביומטרי אישי. הווידאו לא עוזב את הדפדפן, והמצלמה נדלקת רק אחרי הסכמה.

רעיונות לתוספות וכיוונים (לא חובה)

אלה רעיונות שאפשר להיעזר בהם אם בא לכם, לא רשימת דרישות. חלקם גם מזכים בבונוס בבסיס המשותף (ראו בפרק הניקוד). הפרויקט הסופי יהיה גדול יותר בהיקפו: שם תהפכו את האתר לשלכם, למשל באימון מודל משלכם או בשינוי ההיקף של האתר. פרטים על כך יוסברו בכיתה, ואפשר להתחיל לחשוב על כיוון כבר עכשיו.

כיוון	הרעיון	מה הופך אותו לייחודי
כמו FocusFlow המקורי	סרטוני YouTube כרשימת נגינה, עם תמלול שנשלף אוטומטית, ושאלות שנוצרות מהתמלול (למשל עם Gemini) כשהקשב יורד	איכות השאלות, התאמה לרמת הצופה, טיפול בשגיאות של ה-LLM
אתר קריאה	התוכן הוא טקסט ארוך או דף אינטרנט שהמשתמש מוסיף. הקשב נמדד לאורך הקריאה, והאתר מסמן איפה הקורא איבד קשב	אין תמלול מוכן, אז האתגר הוא לדעת איפה בטקסט הקורא נמצא (גלילה, מיקום)
משוב אחר	במקום שאלות: הצעה להפסקה, סיכום קצר, חזרה לקטע שהחמיץ, או פומודורו אדפטיבי	בוחרים איזה משוב באמת משפר קשב, ומודדים
אנליטיקה	דף שמראה קשב לאורך זמן, בין סרטונים וסשנים, מפת חום על התוכן, ומשקלי Attention של v4_2 ומעלה	קוראים במערכת את המודל ומסבירים מה הוא רואה
עבודה עם המודל	השוואת הגרסאות על נתונים שאספתם, החלקת הציון, בחירת סף להתערבות, או, למי שרוצה אתגר, הרצת המודל בשרת עם WebSocket	מדידה והשוואה במקום אמונה
מודל משלכם	איסוף נתונים (למשל הקלטות עם OBS), אימון מודל על נקודות פנים של MediaPipe וייצוא ל-ONNX, עם אותו קלט `[1, 100, 478, 3]` אם רוצים שיחליף את המודלים שקיבלתם. אפשר גם למדוד משהו אחר מקשב, למשל עייפות או מצב רוח	מודל שאתם בעצמכם אימנתם, ובדיקה שלו מול המודלים הקיימים
היקף אחר לאתר	להפוך את האתר למוצר אחר: מאמן ריכוז לעבודה, דשבורד למורה שרואה קשב של כיתה (בהסכמה), לימוד שפה, קריאת קוד או משחק ללמידה	הגדרה חדשה של מי המשתמש ומה הוא מקבל

כללי יושרה

- מותר להיעזר בחומרי השיעור, בתיעוד, בחיפוש באינטרנט ובכלי AI.

- עליכם להבין ולהיות מסוגלים להסביר כל חלק בפרויקט שהגשתם, ואת הסיבות לבחירות שעשיתם.
- אסור להעתיק פרויקט של סטודנט אחר.
- מפתח או סיסמה אמיתיים (Gemini, או סוד של סביבה אמיתית) שהועלו ל-GitHub או נכללו בקובץ שהוגש הם עבירה חמורה ויובילו להורדת ציון. אם זה קרה, החליפו את המפתח מיד. בהגשה מפתח Gemini נשאר ריק, וכל סוד נכנס דרך משתני סביבה, בקובץ `.env` שמופיע ב-`gitignore`..

בהצלחה!